

RUMORS

Persistent replicas and causal iteration

Design and implementation plan

Proposal for review · 8 September 2026
Four chapters and a comparison appendix

Immutable storage. Library-owned snapshots and reference counts.
Concurrent preparation with bounded publication work.

Contents

Overview	4
Reading order	4
Scope and evidence	4
1. Purpose, model, and guarantees	5
1.1 The library today and the proposed revision	5
1.2 Four distinctions to establish first	5
1.3 The guarantees	7
1.4 What is preserved, and what is allowed to cost more	8
2. Storage and ownership	9
2.1 What the engine supplies	9
2.2 Immutable objects	9
2.3 What a reference count means	11
2.4 Construction pays for its own edges	12
2.5 Cheap handles without an unbounded decoded graph	12
2.6 Snapshots are owned by the library	13
2.7 Drop, cancellation, and root cleanup	13
2.8 Zero counts drive reclamation	14
2.9 Bodies can be counted too	15
2.10 Recovery and record validation	16
3. Index, commits, and identity	17
3.1 Give each admitted message a stable local position	17
3.2 Where the within-group order comes from	18
3.3 Keep final records immutable; spend temporary I/O explicitly	18
3.4 Prepare a complete result against one base	19
3.5 Publish with a constant number of ownership changes	20
3.6 Concurrent sessions and retries	20
3.7 Local sends are not remote merge results	21
3.8 Why the index order is causal	22
3.9 Observers and snapshots	23
3.10 Identity transitions need their own durable ordering	23
3.11 Wire and compatibility	25

Contents — continued

4. Implementation and verification	26
4.1 How to use this implementation plan	26
4.2 Package 0: pin the obligations and baseline	26
4.3 Package 1: byte storage and the ownership layer	27
4.4 Package 2: immutable content storage and reads	27
4.5 Package 3: the causal index and immutable placement	28
4.6 Package 4: concurrent publication and local mutation	28
4.7 Package 5: identity, opening, and the wire	29
4.8 Package 6: public observers and lifecycle API	30
4.9 Package 7: a real store, bounds, and operational documentation	31
4.10 Resource accounting and knobs	32
4.11 Decisions that remain measurements or review choices	33
4.12 Source map	34
Appendix A. Review of the earlier proposal	35
A.1 What this plan keeps	35
A.2 Reference accounting belongs in construction, not publication	35
A.3 An owned graph needs roots beyond public snapshots	36
A.4 The proposed cache can keep the whole tree resident	36
A.5 Body counts are not inherently a large-commit bottleneck	36
A.6 The staged-body cleanup order leaks bodies	37
A.7 A set of pending decrements loses multiplicity	37
A.8 A publication binding must outlive old readers	37
A.9 The retry argument overlooks frontiers and candidate scans	38
A.10 Durable identity changes cannot wait for session completion	38
A.11 Causal closure and exact resume need precise statements	38
A.12 Several performance claims should become experiments	39

Overview

Status: design proposal for review. Written and reviewed 2026-09-08 against repository commit `eb224bc35ef809fae74d97554b924e4e6a7d124b`.

This plan adds persistent storage and causal iteration to Rumors. A replica should hold more messages than fit in RAM, recover its state and identity after a crash, and yield causes before their effects without sorting the whole live set in memory. Large updates must leave other synchronization free to proceed.

The design stores immutable objects and prepares new replica states concurrently. Rumors manages their reference counts, snapshots, and cleanup; the engine supplies ordered bytes and small atomic write batches. A finished state becomes current by replacing a small root record. Ownership is recorded as objects are built, so that final replacement has no update-sized backlog of reference-count changes.

The explanation separates three questions:

1. Which messages are live, and what history has the replica observed?
2. Who may issue future events under each part of the peer's identity?
3. Which stored objects must remain available to a reader or unfinished job?

The chapters introduce these concerns separately, then show how they fit together. They assume familiarity with Rust and basic concurrent programming; the library's causal and storage concepts are introduced as needed.

Reading order

1. Purpose, model, and guarantees: the foundations, including what a durable cursor does and does not promise.
2. Storage and ownership: immutable objects, references, snapshots, bounded construction, and reclamation.
3. Index, commits, and identity: causal ordering, concurrent reconciliation, publication, restart, and identity transfer.
4. Implementation and verification: ordered work packages, acceptance criteria, API sketches, and decisions to validate.
5. Appendix: review of the earlier proposal: the decisions retained, specific failure cases, and the reasons for changes.

The first four chapters are a standalone implementation plan. The appendix restates the earlier proposal's relevant choices before reviewing them, so it can also be read without that document. Section 4.11 collects the choices that still need measurements or review.

Scope and evidence

Immutable storage, library-managed ownership, and bounded write batches are requirements throughout. The design requires neither engine snapshots nor locks held through whole-update preparation. Sharding, a durable history log, and tolerance of dishonest authorized peers are outside its scope.

Names and signatures are proposed APIs. Wire-format changes require the repository's owner review and deliberate snapshot-test updates. Source links identify the code inspected; the appendix records the design's provenance.

1. Purpose, model, and guarantees

1.1 The library today and the proposed revision

Rumors replicates a set of messages. Each peer holds the whole live set, changes it locally, and reconciles with other peers through gossip sessions. Reconciliation combines new messages and honors redactions, so communicating peers eventually agree. The peers are authenticated and authorized by the application and are assumed to follow the protocol.

The API separates identity from everyday use. Peer is the unique handle for joining or leaving a universe. It can be exchanged for freely cloneable Rumors handles that send, redact, observe, and gossip concurrently. A Snapshot is a fixed view of the set. Today the set lives in memory; Bookmark stores identity checkpoints but no message bodies. Bookmark recovery rejoins the network to recover content and reclaim identity.

A persistent Rumors replica should keep a live set larger than RAM, restart with its identity and content intact, and iterate messages in causal order without first collecting the whole set in memory. It should still reconcile with other replicas while an unrelated session prepares a large update.

The existing Merkle trie remains the reconciliation structure. A **trie** is a tree whose branches follow successive parts of a key; here, each step follows one byte. A **Merkle** trie also summarizes each subtree with a hash, called its digest, so peers can skip regions where they agree. Its keys are hashes of message versions; that makes them useful for comparing sets but unsuitable for causal iteration.

Add a second trie whose keys represent the order in which this replica admitted messages. Store both tries as immutable objects. A new version of a trie reuses unchanged objects and allocates replacements along changed paths. The library preserves old objects while any snapshot or unfinished operation needs them, using reference counts.

The **root** is the top node from which the rest of a tree is reachable. Replacing its reference can select an entirely new tree while readers retain the old one. **Publication** is the atomic change that selects the new roots and associated peer state. Work prepared before publication remains private.

The engine remains a byte store. Rumors owns the object graph, consistency of its two indexes, snapshots, identity recovery, and reclamation. No engine snapshot, multi-statement transaction, compare-and-set, or merge operator is required.

1.2 Four distinctions to establish first

A message's version is its identity

An interval tree clock assigns a version to each send. It records which earlier events the sender knows, rather than a wall-clock time. Write $a < b$ when b includes a and contains additional history. These versions have a partial order: two independent sends can be incomparable.

Causal iteration respects this order; it does not need to agree on the order of independent messages.

The content trie uses $H(\text{version})$, the hash of the version, as a message's address. Payload bytes do not enter that address or the Merkle digest. Two different payloads must therefore never be published under the same version.

A storage checksum has a different job from a Merkle digest: it detects damaged record bytes, including payload bytes and stored summaries.

A live set is more than its remaining messages

Represent the replica's logical state as $(\text{network}, \text{frontier}, \text{live messages})$. The **frontier** is the version describing all history the replica has incorporated, including redactions. It survives even when no messages remain.

Suppose Alice once held m , then redacted it. Her empty content tree alone cannot distinguish that state from never having heard of m . Her frontier does: when Bob offers m , Alice already knows its version and lacks the message, so reconciliation honors its deletion.

Persisting a newer frontier with an incomplete older tree can consequently turn missing data into apparent redactions. Publishing a frontier, content root, and index root is one atomic operation.

Peer identity grants authority to create versions

A Party owns part of the identity space: the clock's shared space of authority to create events. A **universe** is the set of peers descended from one seed. Its live parties must be disjoint. Forking gives part of that authority to another participant; retiring transfers it away.

A saved tree is harmless to copy as a read-only value. A saved party is not harmless to reactivate: another process may already be using it or one of its descendants. Durable identity transitions therefore need a separate argument from content atomicity.

Exactly one seed creates a universe. Opening an existing store never calls `seed`, and a retired store is distinguishable from an unused store.

Storage reachability is not message liveness

A redacted message is absent from the current set. Its bytes may still be needed by an older snapshot, a session's starting view, or a prepared result.

Reachable means an object can be followed from an owned storage root. **Live** means a message belongs to the current committed set. Reclamation uses reachability, not a search through current message membership.

No per-message redaction record is added to the replicated state. Temporary reclamation records describe unfinished storage work and disappear when that work completes. They are not information peers need to honor a deletion.

1.3 The guarantees

Atomic, durable local state

After a successful persistent mutation, the published state is durable. After a crash during publication, opening yields either the previous complete state or the new complete state. An acknowledged durable state must not be rolled back by the storage implementation.

Publication includes the content root, causal index root, frontier, applicable identity transition, and local publication number. No reader observes only some of them.

An interrupted call can have committed without returning its result. Atomic storage does not remove that uncertainty. In particular, retrying a send after an unknown outcome can produce two messages; application-level request identifiers are needed if the application wants to recognize that situation.

Concurrent preparation, bounded publication work

Tree walks, sorting, index preparation, and retries run without a shared write lock. One mutex, the **ownership gate**, serializes bounded batches of object and reference-count writes and the final root replacement. It never covers a whole walk or import. A separate brief **view lock** lets a reader capture the current in-process roots while a publisher replaces them.

This is a bound on work, not a deadline: one storage write may itself be slow. The engine must eventually complete an accepted operation for the system to make progress. A large update still consumes disk bandwidth and can contend with other work, but it cannot hold the gate for its entire preparation.

Causal iteration

If both a and b are yielded, and $a < b$, yield a first. Concurrent messages may appear in different orders at different replicas. Redacted messages need not be yielded.

Default snapshot iteration, causal ranges, and the causal observer use this order. Keep an explicitly named unordered traversal for compatibility and specialized uses.

Local cursor recovery

A cursor identifies a position in one replica's admission order. When an application resumes from a position it has saved, messages at or before that position are not yielded again; redacted entries can be skipped. Reopening the same, current store preserves this meaning.

That is not exactly-once application processing. If an application performs a side effect and crashes before saving the resulting cursor, it can repeat the side effect. Saving the cursor first can skip the side effect. Rumors cannot atomically coordinate arbitrary application storage through this API.

Return the position associated with each delivery, so an application can save it with its own transaction when that is possible. Keep the existing portable Version checkpoint as a separate, conservative resume mechanism. It can be used on another replica in the same universe and can replay.

Memory and storage limits

Persistent operations must not accumulate a collection proportional to the set or delta in RAM. Traversal stacks, sort buffers, write buffers, cache residency, and active work are bounded explicitly.

Some memory costs depend on individual values: a payload, a version, and the decoded application value. Versions can grow with identity fragmentation. A user-defined deserializer can allocate much more than its input size. A fixed tree depth does not bound those allocations.

State memory as configured working space plus these value-sized costs; provide typed resource-limit errors when configured byte ceilings would be exceeded. The process's total resident memory also includes application values of type T and the storage engine's own allocations. The implementation plan distinguishes these costs and specifies what to measure.

Likewise, old snapshots and unfinished sessions can retain substantial disk space. Bound their number and scratch usage, expose retained-work statistics, and document that retaining old snapshots delays reclamation.

1.4 What is preserved, and what is allowed to cost more

Preserve the content trie's canonical shape: equal sets produce the same tree, regardless of insertion order. Its 32-byte addresses bound it to 32 levels with at most 256 children per branch. This number of children is the branch's **fanout**. Single-child paths are compressed into one stored prefix. Peers compare corresponding address prefixes, using the existing digest and frontier rules and the existing stream schedule.

The causal index adds ordering metadata to supplied messages, explained in chapter 3. That changes byte accounting but adds no network ordering pass.

The causal index necessarily adds storage and mutation work. Removing many messages may require visiting their index entries and eventually reclaiming their bytes even when the wire can communicate that deletion compactly. Do not confuse divergence-proportional communication with constant local deletion work.

Preserve the synchronous memory implementation and its payload cache unless measurements justify a specific change. Its extra causal-index cost must be measured separately from any abstraction overhead.

The difficult limits remain explicit: storage faults can stop progress, perpetual publication conflicts can starve optimistic work, and a lost identity handoff can strand identity space. None should be disguised by a whole-update lock, a silently weakened storage contract, or a recovery guess.

2. Storage and ownership

2.1 What the engine supplies

Keep the contract small. These are capabilities, not final Rust signatures:

```
get(key) -> optional bytes  
scan(lower, upper) -> ascending, incremental key/value stream  
write(batch) -> atomically replace/delete the named keys, durably on success
```

The library uses tagged key spaces. An adapter may map tags to column families or prefix them onto keys. Neither representation changes semantics. The store is exclusively opened by one Rumors instance; the application must not edit its key spaces behind the library.

Required details:

1. `get` returns one complete record. It agrees with completed writes.
2. A batch is atomic across all key spaces. Successful writes are durable. Recovery after an interrupted write exposes either its complete before state or its complete after state.
3. A batch is a bounded in-memory value. Larger preparations consist of many such batches; only the final publication changes the current replica.
4. Scans are bounded-memory, ordered, and support resuming after an exclusive key. They need no snapshot isolation. An immutable, owned range is stable; a changing maintenance range is visited with resumable passes, not treated as an atomic view of the store.
5. The adapter documents error and cancellation outcomes. It must settle or fence outstanding writes before handing storage to a new exclusive opener. A write from an abandoned instance must never arrive after recovery.
6. Ordinary reads of immutable records can proceed independently of the library's ownership gate. No adapter may retain an unbounded request queue or require a session to finish before an accepted storage request finishes.

Do not require “error means nothing committed.” A backend may lose the completion result after committing. Represent a definitely aborted write separately from an unknown outcome when the adapter can distinguish them. An unknown ownership/publication outcome makes the current instance unusable until the backend has settled its writes and the library has reopened it.

Temporary storage is separate: bounded buffers, ordered external sorting, incremental scans, and explicit cleanup. It needs no crash durability because no published object depends on temporary bytes. The caller can supply files or an appropriate engine key space. This facility is not an engine snapshot.

2.2 Immutable objects

Use sequential, never-reused object IDs. IDs are storage identities, not message identities and not Merkle hashes. Reserve ranges durably with a small allocator update; unused IDs after a crash are harmless holes.

An object is immutable from the moment its complete record is installed. Objects contain IDs of their dependencies, not recursively cached decoded children. A content branch and a causal-index branch each have at most 256 children. An **owning edge** is a stored reference that keeps its target allocated. Together, the objects and these references form an ownership graph.

The ordering metadata has two parts: which operation admitted a message (its **admission group**), and its position among that operation's messages. A received candidate also carries a **sender tag**, its position in the sender's order. Chapter 3 explains how the receiver uses these to assign a new local order. Start with these object kinds:

Object	Immutable contents	Owning edges
Body	Exact admitted payload bytes and their length	None; a chunked representation can add bounded-fanout body objects later
Received candidate	Version, body ID, sender tag; private to reconstruction	Body
Message	Version, body ID, admission-group ID, position within that group	Body and admission group
Content leaf	Compressed path and message ID	Message
Content branch	Compressed path, children, child digests, summaries	Children
Index leaf	Compressed local-order path and message ID	Message
Index branch	Compressed path, children, version span, count	Children
Admission group	Publication number assigned to the group	None

The shared message object is useful: changing a leaf's compressed prefix does not duplicate its payload or its placement metadata. The index points to the same message object rather than a particular content-leaf wrapper. Index correctness is equality of message membership, not equality of wrapper pointers.

The graph must have no cycles: branches reference lower tree structure, leaves reference message records, and messages reference bodies and groups. Nothing points back from a body or group into either tree. Reference counting alone would not reclaim a cycle. Temporary reconstruction nodes obey the same rule, with received candidates in place of final messages.

A branch stores the summaries needed without descending: hash where applicable, leaf count, version floor and ceiling, and maximum version-bound size. The floor describes history common to every leaf; the ceiling contains every leaf's history. Together they are the branch's **version span**, used to accept or reject a whole subtree during a causal query or merge.

This summary is distinct from the replica's frontier, which survives deletion. Summaries are computed at construction. Persist child digests inline in content branches so supplying a child listing does not read every child.

An index leaf initially reads the version from its message object. It does not copy potentially large version bytes merely to avoid that metadata read. If measurement favors an inline version, that is a record-layout decision; either layout keeps bodies out of index-only queries.

Admission groups need a special construction reservation because their publication number is not known during preparation. Section 3 explains the one-time binding. No published graph reaches an unresolved reservation.

2.3 What a reference count means

For each allocated object x , maintain:

```
refs(x) = number of owning object edges to x
         + number of durable root entries owning x
```

Count **all** allocated objects, including private preparations and objects waiting to be reclaimed. The equation does not depend on whether a node has ever been published. Each occurrence of an owning edge counts, even if two edges happen to target the same object.

A durable root entry is a small record saying that some operation or replica view owns an object. Root entries include:

- the current replica's content and index roots;
- prepared roots and temporary construction roots;
- roots retained for older snapshots and session views.

A private root entry is called a **hold**. It protects work that is not yet reachable from the current replica, such as a branch still being built.

The key spaces can initially be:

Meta	format, store ID, allocator state
Head	roots, frontier, identity state, publication number
Objects	immutable records, by object ID
Refs	reference counts, by object ID
Holds	private roots, by operation ID and hold ID
Retained	older views still represented by durable roots
Zero	zero-count objects pending deletion
GroupReservations	groups awaiting their publication number
IdentityReservations	recoverable, unsent party portions
BodyDirectory	optional address -> shareable body ID

Use checked arithmetic for counts and IDs. Overflow is a typed refusal before mutation. The count is not inferred from live messages or recomputed by walking the set during normal operation.

2.4 Construction pays for its own edges

Register an object's owning edges when constructing it, not when publishing the completed replica. Construction and its count updates form one atomic ownership transaction.

For a new branch with children a, b, and c:

1. Hold valid read ownership of every child throughout the operation.
2. Outside the ownership gate, compute the complete immutable branch record and its summaries.
3. Under the gate, read the required counts, install the branch, register its three child edges, initialize its count to one, and create its construction hold. All of these changes are in the same atomic batch.
4. Only after success, make the new owned handle available to other work.
5. Release construction holds that the builder no longer needs. Their child references now survive through the new parent.

A whole bootstrap repeats this operation incrementally. Every intermediate graph is owned and recoverable, but no intermediate graph is current.

Batch adjacent constructions up to fixed byte and operation limits. A batch can combine increments and decrements to the same child; it must preserve their multiplicity. Provisional handles whose objects are still inside the batch cannot escape to another task. Flush before yielding such handles.

The gate can cover several prepared records and a bounded number of count reads, never an entire received region. Its maximum work is a configuration parameter plus the cost of the largest allowed record. Do not make it depend on a session's number of messages.

This also solves the “existing children” problem. A new parent sharing an old subtree increments that subtree's count now, while the source view still protects it. The final publication owes no deferred increment for that edge.

Durable writes at this frequency have a cost. Begin with bounded coalescing, measure storage barriers per admitted message, and tune batching. Do not add a weaker durability mode until its ordering and recovery rules have their own specification. The initial contract is intentionally sufficient by itself.

2.5 Cheap handles without an unbounded decoded graph

Separate three things:

ObjectId	identifies immutable storage
OwnedRoot	keeps a storage graph alive
LoadedNode	temporarily holds decoded bytes while a graph owner protects them

Cloning OwnedRoot shares an in-process Arc around one ownership claim; it does not increment a persistent count. Traversing a child keeps the source root owner alive. It need not create a new durable root for every read. Creating a new parent is different: its child edges are real persistent ownership and are registered by the construction transaction.

Decoded branches contain raw child IDs. They do not each contain a `OnceLock<Arc<DecodedChild>>`. Otherwise a root touched by a full scan retains the entire decoded tree even after the cache evicts its least-recently-used entries.

Cache values may be Arcs. Count the bytes retained by cache entries and by loaded nodes still held outside the cache. Eviction only releases the cache's share. An operation's live loaded-node set is bounded by the traversal/window budget. A cache hit does not substitute for a storage ownership claim.

`read` returns an owned decoded guard, not a reference borrowed indefinitely from an evictable cache. A user-visible message owns its decoded payload after the body has been read; merely keeping that `Arc<T>` need not pin storage.

2.6 Snapshots are owned by the library

The current in-process replica view contains an `Arc` that names the two roots and their frontier. Taking a snapshot clones this view under the same brief synchronization used to publish a replacement. No storage I/O is needed.

When replacing the head, the publication batch transfers the old head's two root references into a `Retained` record. It transfers the prepared roots' holds into the new head's ownership. These are ownership transfers, so their net count changes can be zero; they must still be represented atomically.

The old `Retained` record remains while any in-process view needs it. This includes explicit snapshots, a session's starting view, a prepared merge's base, and a reader with I/O in flight. An internal view uses exactly the same lifetime mechanism as a public snapshot.

Once the final in-process view disappears, its retained root record can be deleted and the roots decremented. The library needs only a bounded registry of active views and construction holds, not an entry per reachable node.

The publication transition and view capture must be coordinated: a reader either acquires the current view before the swap, in which case retention already protects it, or acquires the new view. It cannot obtain a bare old pointer in the interval between publication and reclamation.

Registry limits apply to distinct views and operations. Clones of one view share its registration. If retaining another published generation would exceed the configured limit, publication can wait or return a resource-limit result; it cannot invalidate a snapshot already handed to the application.

2.7 Drop, cancellation, and root cleanup

Rust `Drop` cannot await. Give storage-owning handles an explicit asynchronous release, and keep synchronous drop safe without allocating an unbounded queue of work.

Every private construction hold already has a durable `Holds` row. The in-process registry records whether that hold is active. Synchronous drop removes its active registration. The row remains

until bounded maintenance deletes it and decrements its object. Maintenance resumes scans by key and revisits active rows on later passes.

Temporary files can also contain object IDs awaiting a later pass. Those IDs need ownership even though no in-memory handle exists. Support a **parked hold**: a durable root row retained by the operation as a whole, rather than by an individual in-process handle. One active-operation registration protects all of its parked holds. Their inventory is on storage, so parking a million candidates does not create a million registry entries in RAM. Reading one borrows that operation's protection; explicitly consuming the hold releases it only after the receiving parent or another owner is registered.

Use parked holds for candidate IDs in sort files and final message IDs waiting for the address pass. They cost storage proportional to unfinished work and bounded ownership updates to release. If an operation ends or crashes, its remaining parked holds become eligible for a resumable cleanup scan. Never treat a bare ID written to nondurable scratch as an owning edge.

Normal asynchronous builders release their exhausted holds promptly, keeping the number of active roots bounded by their working frontier. Before accepting more construction, apply backpressure if root registrations or cleanup debt reach their configured limits. Do not let a per-session vector of abandoned handles grow with the set.

Cancellation during a store write is an ownership outcome, not just a dropped future. If its result is unknown, a synchronous guard marks the instance as needing recovery before the gate can be reused. The adapter must settle its outstanding I/O before reopen; recovery then reads which complete batch exists. Cancellation between settled batches merely abandons durable private roots.

2.8 Zero counts drive reclamation

Zero is a set of objects whose count is **already zero**. It is not a set of pending decrements. This distinction prevents losing two decrements to the same object by storing them under one key.

Removing a root or an object edge decrements its target in the same atomic batch that removes the edge. If the count becomes zero, add the target to Zero. Zero-count objects cannot acquire new owners: construction must use an existing valid graph owner, and a directory lookup must acquire ownership under the gate before returning a shareable object.

For each zero-count object, reclamation:

1. Reads its immutable record to learn its outgoing edges.
2. Under the gate, rechecks its zero-count work entry.
3. Deletes the object, its count, and its work entry; decrements every outgoing edge, enqueueing any newly zero targets, all in one bounded batch.

Another collector may consume the work entry during step 1. If the record has disappeared, recheck under the gate: a missing work entry means the work is finished, not corruption. A group reservation has no outgoing edges and can be reclaimed without an ordinary object record.

The count equation remains true even while the cascade is unfinished: a zero-count parent retains its child edges until the parent is deleted. Atomic queue consumption makes replay after a crash safe.

The batch limit must accommodate at least one maximum-fanout object's edge updates. If a representation cannot meet that limit, give its reclamation an explicit persisted progress record; do not perform half of a decrement set and hope to infer the rest after a crash.

Offer bounded maintenance after mutations and an explicit `reclaim().await` that drains currently eligible work. A successful send need not await an arbitrarily large unrelated cascade. A caller may drive reclamation more aggressively, and storage-pressure backpressure must eventually require it. The library remains runtime-independent; no hidden task or timer is required.

Deletion makes space reusable; it need not shrink the engine's file or securely erase historical media bytes. Those are separate engine operations.

2.9 Bodies can be counted too

Count a message object's edge to its body when constructing that message. Reclamation then releases the body through the same graph algorithm, without checking membership through every snapshot.

Body sharing is an optimization layered on this correct lifetime model. For incoming, already-authored messages, a directory maps the version address to a body object. Acquire a temporary body hold under the ownership gate, then build a message pointing to it. Competing sessions select the same directory entry. Any privately written losing copy is ordinary reclaimable work. This guarantees safe sharing; avoiding every duplicate transient write is a separate performance goal.

When a directory body's count reaches zero, remove its matching directory entry in the same ownership transaction. The directory is not itself an owning edge, or it would keep every body alive forever. IDs are never reused, so a replacement body cannot be confused with an old queued deletion.

Local sends require special care. Two speculative local batches can compute the same next version for different values before one wins publication. Their private bodies must use independent object IDs; they cannot enter a version-address deduplication directory. A conflict requires replaying the losing local intent with new versions, not joining these two drafts. This distinction is specified in section 3.

One immutable message can be referenced by both content and index leaves. Consequently, a redaction removes both references, while old views retain their own paths to the message and body. There is no per-body snapshot scan and no special body-liveness marker sweep.

2.10 Recovery and record validation

On exclusive open:

1. Ensure previous writes have settled; validate format and head metadata.
2. Establish ownership of the current roots before serving readers.
3. Reconcile the identity reservation state as specified in section 3.
4. Treat private holds and retained views from the previous process as abandoned. Remove them in bounded ownership transactions.
5. Resume zero-count reclamation. It can run incrementally; expose remaining recovery work instead of requiring a full live-set scan before service.

Creation installed each object (or group reservation), its edges, its count, and a root hold atomically. Therefore recovery need not enumerate allocator chunks looking for uncounted objects. Every abandoned allocated graph has a durable owner record, or is already on the zero-count cascade. Unused allocated IDs contain no records and need no cleanup.

Records carry a kind, format version, and checksum over an unambiguous encoding of key space, key, and contents. Lengths and arithmetic are checked before allocation. Decode errors identify the key and failure. Never turn a missing required object into an empty subtree.

Validate path compression, child ordering, bounds ordering, and parent/child digest agreement at the appropriate read boundary. A checksum and an ordered span do not prove a stored memo equals the fold of the children. Constructors, property tests, and an explicit full audit provide that stronger assurance. The audit is useful maintenance, not a mandatory open traversal.

CRC-32C is a reasonable initial fault-detection choice if its accepted residual risk is documented; no checksum detects every corruption. Do not base the choice on an unmeasured universal throughput number or claim Merkle hashes protect payload bytes.

3. Index, commits, and identity

3.1 Give each admitted message a stable local position

Give the index a two-part key:

```
(publication number, position within an admission group)
```

Both fields are checked u64s, encoded in big-endian order when used as index paths. The resulting trie has at most 16 byte levels. Publication numbers strictly increase with every head change, including an identity-only change. This number is also the head's **revision**, used to detect a changed base during publication. Positions inside a group are unique. Neither counter wraps. A redaction removes an entry but never makes its position available for reuse.

An **admission group** contains the candidates prepared by one operation. Assign their within-group positions in causal order before attempting to publish. Candidates later rejected by reconciliation leave holes. Dense numbering is useful for initial construction, but it is not an invariant worth rebuilding surviving records to preserve.

The publication number is assigned only when the operation wins publication. Two groups prepared concurrently therefore do not have a predetermined order. This preserves the freedom to prepare and finish them independently.

A message record stores (group ID, within-group position). A small, immutable group object supplies the publication number after commitment. Looking up the group is normally cacheable, but price it as a possible read. The group object is retained through actual references from message objects, including messages kept by older snapshots. It is not deleted merely because its last *currently live* message was redacted.

Binding a group once

Reserve a group ID before building its message records. The reservation is durable construction metadata with a count and an owning hold; it is the one permitted unresolved dependency in a private graph. It contains no message list and has no outgoing object edges.

At successful publication, atomically install the immutable group object { `publication_number` } and remove its reservation. No existing object record is overwritten. If no messages from that group survive, publish no binding; releasing the private graph eventually releases the reservation.

Type the boundary: a prepared root can contain a reserved group; a published root cannot. Preparation establishes that the only unresolved group it reaches is its own. Current subtrees already satisfy the published invariant. Publication resolves that single group in constant record-count work.

This small reservation is necessary because the publication order cannot be known while immutable message records are being prepared. It is not a transaction over the group's messages.

3.2 Where the within-group order comes from

For local sends, use the order in which the batch issues sends and ticks their versions. Redactions do not create index entries.

For gossip, each supplied message carries the sender's existing local key as a 16-byte origin tag. A session has one remote sender, and every outbound tag must refer to that sender's fixed, committed session view. The receiver sorts the tags in temporary storage and assigns its own within-group positions. It does not retain the previous hop's key as part of its own key.

Example:

```
sender's keys:      (8, 2) (8, 9) (11, 0)
receiver's group:   0      1      2
receiver publishes: (27, 0) (27, 1) (27, 2)
```

The sender's order puts every included cause before its effect. Removing items from an ordered sequence preserves that property among the survivors. The receiver can therefore filter candidates without sorting them again.

The wire still sends leaves in address order, which supports efficient content-trie construction. The extra tag is metadata on those existing supplies. It does not add a causal-order wire phase or additional round trips.

Audit all supply paths, including opening supplies and terminal leaves, to ensure their origin is the starting committed view. Private received nodes with unresolved group bindings are never an outbound source. The existing session contract that supplies belong to the sender's declared set provides the starting point for this audit.

Validate tag uniqueness across a session, not merely inside one run. A sort implementation must detect duplicate keys rather than silently replace one message with another. As with the existing conformance checks, this detects bugs in authorized peers; it does not defend against dishonest senders.

3.3 Keep final records immutable; spend temporary I/O explicitly

The received payload and version are available before their local position. Do not put a placeholder position into an ordinary content leaf and expose that leaf to caches or other builders.

Use two representations:

- A private received candidate refers to a validated version and owned body. The session's temporary reconstruction uses these candidates alongside handles into its starting view.
- A final immutable message record has the version, body, group ID, and within-group position. Content and index leaves refer to this record.

After the wire reconstruction finishes:

1. Scan candidates sorted by origin tag, assigning unique positions. Store (address, final message ID) in a second temporary ordered stream.

2. Scan that stream in address order to build final received content subtrees bottom-up and substitute them into the session result. Store the list of reconstructed trie prefixes and the temporary nodes incrementally too, rather than accumulating them in RAM.
3. Release temporary candidate/reconstruction roots as they become redundant.

Every object ID in either temporary stream remains protected by its operation's parked holds or by a known retained graph until that stream has been consumed. The ownership layer distinguishes these from dropped transient handles. Sorting object IDs does not itself keep their objects alive.

This is an additional metadata pass and some additional node construction. It does not re-decode or recopy every body. Unchanged subtrees from the starting view remain shared.

The implementation can alternatively represent the reconstruction as an immutable edit description and materialize its final nodes once. That is an optimization only if it stays simpler than building the temporary tree. Start with the auditable two-representation path and measure its cost.

An **external sort** orders data larger than memory: sort one bounded buffer at a time into temporary files, called runs, then merge those sorted runs. Limit the total buffers, merge readers, output buffers, and open files across both sorts. If there are too many runs to merge at once, use repeated passes; two passes are not a general bound on arbitrarily large input. Keep the run inventory on disk too. Enforce temporary-byte and concurrent-session limits before accepting more work.

3.4 Prepare a complete result against one base

Let B be the current replica view captured by the operation. Let S be a completed session result with final immutable messages and its frontier.

Outside the ownership gate:

1. Compute $P = \text{join}(B, S)$ using the same deletion rules as $\text{Tree}::\text{join}$.
2. Preserve B's message records wherever both inputs contain the same version. Equal hashes compare content; they do not make different local group IDs interchangeable. This preference preserves existing index keys.
3. A received candidate is new exactly when it exists in P and is absent from B. This is a membership test, not a test of pointer identity.
4. In candidate order, build an index subtree for the admitted messages bottom-up from the sorted entries. This **bulk build** writes each node once instead of rewriting a path for every insertion. Its within-group positions are already fixed. Place it under the prospective next publication number in a prepared index root.
5. Remove the index entry for every message removed from B by the join. Keep the removed subtree owned until its entries have been enumerated.
6. Prepare the resulting frontier and root summaries, and acquire explicit owning holds for the two result roots.

Every object and edge produced along the way is registered by the bounded construction transactions from section 2. There is no accumulated final batch of reference counts, body increments, or index deletions.

A whole-subtree content deletion can be decided without visiting its leaves. Updating the other index generally cannot: those messages occupy unrelated local-order positions. Enumerate the affected messages incrementally and charge that work to the deletion. If useful, externally sort removals by index key to reduce path rewriting. Do not hide an $O(\text{number removed})$ operation inside a supposedly constant-cost prune callback.

3.5 Publish with a constant number of ownership changes

The finished operation has a base revision, two owned prepared roots, a prepared frontier, and at most one unbound admission group.

Under the gate:

1. Compare the current revision with the preparation's base. A revision includes identity changes as well as content changes.
2. If they differ, release the gate and reprepare. Do not write the head.
3. Otherwise assign the next publication number and bind the admission group if any of its messages survived.
4. Atomically replace Head, transfer its old root ownership into a retained view record, and transfer the prepared root holds into the new head. Include the applicable identity-state change in this same batch.
5. Publish the new in-process view under the brief view lock. A guard handles cancellation or an unknown write result by marking recovery necessary; it never allows memory to continue as though a possibly committed write did not occur.

There are a constant number of root/hold/group records here, independent of the delta. The frontier and party encodings are variable-sized values; “constant number of records” does not mean a fixed byte count for all clocks.

After releasing the gate, release private roots, reclaim bounded eligible work, and dispose of temporary storage. A cleanup failure after a successful publication must not be reported as though the logical commit were rolled back. Keep commit outcome and maintenance status separate.

Hold the old roots and all potentially discarded user values across the in-process swap. Drop them outside the view lock. Keeping the old root alone does not prove every temporary value or failed candidate is protected; retain the existing destructor/panic tests and audit all exits.

3.6 Concurrent sessions and retries

Two sessions can read the same base, receive different deltas, construct objects, and prepare indexes concurrently. Their ownership transactions interleave in bounded batches. Only their final root replacements conflict.

On a conflict, a gossip operation retains its completed session result S and joins it against the new current view. This correctly incorporates newer redactions and preserves already admitted messages' positions.

For the initial implementation, rerun admission as a bounded-memory scan of the candidate stream. Reuse unchanged content subtrees and index subtrees where the algorithm establishes equality. Keep within-group positions fixed; only holes and the prospective publication-number prefix can change.

An optimized implementation can use a structural difference walk between the old and new bases to revisit fewer candidates. That optimization needs a work bound and tests. A tree-identity check inside a loop over every candidate still visits every candidate. Do not claim “overlap-only retry” until the algorithm actually skips unaffected candidate regions.

The causal cases remain straightforward:

Concurrent changes	Required result
Both sessions supply m	First publication assigns its position; the other retains that record and adds no index entry
One supplies a cause, another the cause and effect	The cause is earlier within one group, or belongs to an earlier publication
One supplies m , another honors its redaction	The final join removes m ; its index entry disappears with it
Both remove the same messages	Only entries still present in the chosen base are removed
An old snapshot remains open	Its roots and group bindings remain reachable regardless of the current set

Never respond to repeated conflicts by holding the ownership gate through reparation. That would violate the concurrency requirement. Use a bounded retry-work policy, returning a distinct contention result when exhausted; retain or release the private work according to the explicit retry API. The peer remains usable. This promises progress when publication eventually offers a successful attempt, not starvation freedom under perpetual change.

A fair queue for the short gate prevents one task monopolizing ownership batches. It does not, by itself, guarantee that an optimistic preparation will ever find its base current. If workload measurements show unacceptable large-session starvation, improving rebasing is a required design follow-up; a long exclusive fallback is not an acceptable hidden solution.

3.7 Local sends are not remote merge results

Incoming messages already have globally established versions. Local intents do not. A local batch contains “send this value” and “redact this version,” not a finished independent replica to merge after a conflict.

Admit and serialize values before preparing; keep the resulting intents owned or in bounded temporary storage. Against a captured base, assign draft versions using its current party and frontier, then build the content and index results. Draft versions are private, are not returned to the caller, do not enter shared version-address body deduplication, and never cross the wire.

The captured identity description is inert draft input, not another usable peer identity. Keep any decoding or read-only aliasing needed for the clock calculation inside a private draft type that cannot fork, donate, or transmit authority. A draft tree cannot enter the ordinary peer-to-peer merge surface. Only the successful revision check turns its versions into emitted events.

If publication loses its base, discard the version-stamped draft and replay the intents against the new base. Do **not** rerun the user's closure. In particular, do not merge a losing local draft into the winning state: two draft sends from the same base can have equal versions and different payloads.

The successful attempt assigns versions above the frontier current at its publication. Identity changes also invalidate an attempt, so a send cannot publish using a party portion that was donated while it prepared.

The memory mode can continue assigning versions synchronously in its existing commit critical section. It does not need to adopt persistent draft/retry machinery merely to share an API facade.

A synchronous callback cannot spill an unbounded batch through async I/O. Keep a bounded callback-based persistent batch for convenience and provide an async staged-batch builder for larger atomic batches. Enforce the bound while queuing, before memory has already been consumed.

3.8 Why the index order is causal

The proof needs more than “a cause's version is below the frontier.” Any frontier containing an effect also contains its causal past, even if an incorrect implementation lost the associated bodies. Start from the actual state invariant:

When a message becomes live, every earlier emitted message in its causal past has already been incorporated by that replica, or is incorporated in the same atomic result. An incorporated message that is absent has been redacted and cannot later become live again under that version.

Establish this inductively from local sends, complete reconciliation, frontier-based deletion, and atomic recovery. In the test model, retain an independent history of emitted messages to state it; production needs no history log or tombstones.

Now consider two live messages $a < b$:

1. If both were newly admitted in one publication, sender tag order or local send order puts a before b within the group.
2. If a was already present when b was admitted, its publication number is lower and its position is preserved.

3. a cannot first become live in a later publication than b: the state invariant says it was already incorporated, and if absent as a redaction, it cannot reappear under that version. Thus the index extends causal order. The argument also explains why partial wire results cannot be published just to reduce a large commit's work.

3.9 Observers and snapshots

A snapshot keeps one complete pair of roots owned and traverses its index lazily. Its range iterator uses the index's version-span summaries to prune; its point lookup uses the content tree. All cold reads are fallible and async. Cached root summaries support synchronous `len`, `latest`, and `hash` access.

The live causal observer advances a local cursor through successive views. Each bounded page or in-flight read owns its view. It releases that view before waiting for a later change. A suspended observer therefore need not pin an entire old pass indefinitely.

For a pass, capture a frontier and an upper publication boundary from the same view. Visit entries after the cursor and at or below that boundary. Moving to a newer view may skip newly redacted entries, which cannot return; entries newly admitted by that newer view are above the fixed boundary. The pass completes only when that interval is exhausted.

Advance the portable checkpoint to the captured frontier only after the pass's messages have been delivered under the existing checkpoint convention. Do not substitute the newest head's frontier after a racing end-of-scan: that could cover undelivered messages in a new publication. Do not construct the portable checkpoint by simply folding yielded versions.

A local `Position` contains store identity and a key, with explicit start and end-of-publication boundaries where needed. Prefer opaque constructors and validated serialization. Validate foreign stores, future publications, overflow, and the maximum issued position at the current boundary. Preserve enough high-water metadata to validate a legitimately issued cursor even after its message is redacted. An older genuine cursor need not name a live entry.

Return a delivery's after-position with that delivery. Document saving it after successful handling. A `Position` is not an acknowledgment stored by Rumors, and a `Version` checkpoint is not a substitute for an exact local key.

3.10 Identity transitions need their own durable ordering

Persist Network, store identity, codec-relevant configuration, and lifecycle state with the head. An empty store, an active peer with an empty message set, and a retired peer are different states. A corruption error is none of them.

Local sends are durable before they become available to snapshots or gossip. Ordinary outgoing gossip uses a committed view. This prevents restart below versions this incarnation has already exposed through the library.

Forks and retirements require more: the donor must durably give up authority before the recipient can receive it. Use a small durable reservation state machine, coordinated by the ownership gate.

Donating a fork for bootstrap

1. Atomically capture the session's starting view, split the active party, and record the donated portion as a **reserved**, **unsent** identity. Persist the retained active party and reservation together before using the split. The reservation carries the frontier at the split. Capturing these together prevents intervening local sends from outrunning the donated clock.
2. Reconcile using that starting view. Normal sends use only the retained party. Other sessions can continue; their prepared attempts observe the changed revision.
3. Immediately before attempting to write any identity bytes, atomically remove the reservation from recoverable custody. The active party remains narrowed. Wait for that write to be durably known successful.
4. Send the party. From this point onward, no error, cancellation, or missing acknowledgment can return it to the donor.

If a normal failure occurs before step 3, an async recovery transaction can join the reservation back into the active party. If the process crashes while the reservation is still durably unsent, open can do the same. A cancelled operation leaves its unsent reservation for the next recovery point; synchronous drop cannot perform a durable join.

If step 3 succeeded but no recipient acquired the party, that identity space is stranded. Preserving it through an ambiguous handoff would require a separate transfer protocol. Accept this existing class of loss rather than reactivate possibly duplicated authority.

Retiring

Apply the same sequence to the whole party. A recoverable, unsent retirement reservation permits recovery. Once authority has been relinquished, persist a retired lifecycle state that open cannot turn back into an active peer. Keep the existing recovered/declined/uncertain distinctions in the public lifecycle result, extended with storage outcome where necessary.

Receiving authority

The receiver first completes reconciliation and receives the donated party. It prepares the merge against its current state and validates disjointness. Publish the resulting content, index, frontier, and expanded party atomically before sending the session's completion certificate.

If the recipient crashes before publication, the donated space may be lost. If it crashes afterward, it reopens with both the authority and the state that makes using it safe. The donor never recovers authority solely because the completion certificate was lost.

Only the active identity manager can operate on party bytes. Old snapshots and retained root descriptors do not expose reactivatable parties. Storage copies, backups, and an earlier process incarnation must never operate as independent live peers under the same stored identity.

3.11 Wire and compatibility

Recommend `Protocol::V3` for origin tags, with no compatibility shim for this unreleased library. This is a proposed owner decision. Before implementing it, record the approved vocabulary and deliberately reaccept the wire pins with a commit that names that change.

Tags add bytes to every supplied leaf, including one-message and opening runs. Update run sizing, allocation accounting, maximum fan occupancy, codec fixtures, wire observation/rendering, and the streaming conformance tests. Recheck the deadlock argument with slow storage and bounded ownership batches; never hold the ownership gate while waiting on a network peer or a channel whose consumer needs that gate.

Keep `Bookmark` for the memory mode. A persistent peer's head and identity reservation records take responsibility for durable custody; attaching an independent bookmark to that peer would create two authorities of record. Opening a persistent store restores it directly, rather than rebootstraping as bookmark recovery does.

4. Implementation and verification

4.1 How to use this implementation plan

Complete the work packages in order. A package's acceptance criteria are conditions for advancing, not a list to defer until the end. Keep incomplete persistence machinery private or experimental until crash ownership and identity recovery are tested. Do not ship a persistent mode that intentionally has no reclamation and promise to add its ownership model later.

Before each implementation commit, run `just gate` as required by the repository. Use the `justfile` for narrower iteration. Regenerate READMEs with `just readme` after crate-level rustdoc changes. Every new test has an English doc comment; families of schedules and failure boundaries are proptests. Commit every generated regression seed at its resolved persistence path.

4.2 Package 0: pin the obligations and baseline

Read the crate docs and the modules listed in the source map below at the implementation revision. Confirm how local commits, session snapshots, identity donation, payload decoding, and observers interact.

Write a small state model before broad Rust refactoring. It should include immutable objects and edges, root holds, zero-count work, a current head, private preparations, two indexes, and identity reservations. A test-only history records actual emitted messages for the causal invariant. This model is an **oracle**: an independent calculation of the expected result, used to check the implementation.

Model the following atomic transitions: bounded object construction, hold release, one zero-object reclamation, root publication, group binding, party reservation, and relinquishment. Model a crash between every transition and both outcomes of an interrupted atomic write.

Measure the existing memory mode on representative small and large payloads, different version widths, sparse deltas, bulk bootstrap, redaction, snapshots, and observers. Separate CPU, allocated bytes, peak live nodes, and compile cost. Record workload sizes and measurement uncertainty before selecting an acceptable overhead threshold.

Acceptance:

- The model checks the count equation continuously, not only after cleanup.
- A bounded publication touches no per-message ownership records.
- The model exposes unsafe donation-before-durable-relinquishment and unordered-group publication as failing negative controls.
- The baseline distinguishes content-tree performance from the proposed causal index's necessary extra work.

4.3 Package 1: byte storage and the ownership layer

Implement private Store capabilities, structured error outcomes, allocator, record framing, counts, construction holds, retained roots, and zero-count reclamation. Include the admission-group reservation as an explicit object construction state from the beginning.

Build a test store with separate visible and durable state, atomic batches, fault injection, and a pending-write model. A plain BTreeMap clone alone does not exercise uncertain completion, cancellation, or write settlement. Scans should be able to interleave with unrelated writes without promising snapshots.

Use a synthetic immutable graph with no cycles to exercise ownership before tree integration. Cover repeated edges to the same child, shared roots, cancellation during construction, two concurrent collectors, and dropping root handles while a publication transfers their durable ownership.

Acceptance:

- Each crash recovers a valid graph, and abandoning all private owners leaves exactly the current graph once reclamation drains.
- An incoming edge is never installed without a protecting owner of its target; zero-count objects cannot be revived.
- No count or cleanup update is lost when the same object is referenced more than once in a batch.
- Active-root registration and dropped-hold cleanup remain bounded as object count grows. Parked sort-file references retain their objects through an operation-level registration. No root list in RAM grows with the prepared tree.
- One slow bounded ownership transaction can delay the gate; a whole multi-batch construction cannot retain it between batches.

4.4 Package 2: immutable content storage and reads

Add body, received-candidate, final-message, content-leaf, and branch records. A received candidate is an immutable version/body pair without final local placement; it is private to reconstruction. Add loaded-node guards and a bounded decode cache whose child references are raw IDs.

Adapt the existing session backend boundary to these handles. In particular, the current `Leaf::leaf(version, message)` surface has neither an instance argument nor ordering context. Introduce explicit ingress context for the store, operation owner, codec, and origin tag; do not hide it in thread-local state. The outgoing leaf surface must fetch body bytes asynchronously while keeping source ownership alive.

Implement persistent join, unknown, bulk assembly, point lookup, and an address walk. Reuse pure rules and representation-independent helpers where that removes duplication. Keep the existing synchronous memory traversals; do not turn every memory operation into a boxed future and poll it with a no-op waker as an initial architectural requirement.

Use explicit bounded traversal state where needed for async reads. Retaining the height-typed protocol does not require every storage walk to encode its entire recursive future in the type system.

Acceptance:

- Generate one legal single-universe event history and replay its immutable versioned inputs through both storage representations. Persistent content matches `Tree::join`; the oracle is a shadow calculation, not another live author holding a duplicate party. Never mix independently seeded universes.
- Empty-after-redaction preserves its frontier.
- A held snapshot survives replacements and reclamation; closing it makes exclusive old objects reclaimable.
- A complete scan with a small cache does not retain the whole decoded graph.
- Backend memory pricing includes loaded records and guards, not Local's pointer-only charge.

4.5 Package 3: the causal index and immutable placement

Implement the 16-byte index trie, group reservation/binding, index-only iteration, range pruning, and temporary origin/address sorts. In memory, the corresponding group binding can be a one-time initialization shared by Arc; it does not require an engine or a persistent count.

Implement the candidate-to-final-message conversion and the address-ordered replacement build. Test sparse path-compressed scopes, whole-root imports, and a candidate removed from the reconstructed result before finalization. The temporary order map is replayable for admission retries.

Integrate exact index additions and removals. Keep existing message records when joining equal content, and ensure prefix changes do not change a message's local position.

Acceptance:

- Content and index name exactly the same message set at every publication.
- For generated emitted messages, every yielded cause precedes its effect.
- No published object reaches an unresolved group.
- A group stays readable while only an old snapshot references its messages, then disappears after their last references are released.
- Duplicate origin tags are errors, not overwrites; overflow cannot wrap a position or publication number.
- Sorting exceeds memory and merge-fanout limits in tests and still remains bounded, including its run inventory and output buffers.

4.6 Package 4: concurrent publication and local mutation

Implement prepared-root ownership transfer and head publication. Include storage outcome guards around the durable-write/in-memory-view boundary. Add gossip rebasing against a

newer head and a bounded retry-work policy. Start with a streaming candidate rescan; add selective retry only with an actual region-skipping algorithm.

Implement persistent local mutations from replayable intents. An unsuccessful attempt must discard draft versions and regenerate them; the user closure runs once. Add a bounded callback batch and an async staged builder.

Add body sharing for imported messages using the optional directory and counted body holds. Keep speculative local bodies out of that directory. This can be a separate commit within the package: correctness never depends on eliminating duplicate transient body writes.

Acceptance:

- Force an unrelated publication between preparation and commit for every overlap/redaction case in section 3. The result equals the corresponding serial oracle operation on the new base.
- Two draft local sends with equal versions and different payloads cannot both publish or be merged as equal content.
- A payload destructor that takes a snapshot cannot run under the view lock; panics and cancellation do not expose partial state.
- Publication performs a constant number of ownership-record operations as the update grows. Instrument count reads, count writes, and gate ownership, not just elapsed time.
- During a large preparation and reclamation, small independent sessions continue taking ownership and publication turns. No retry fallback holds the gate through a walk.
- A contention result is distinct from corruption and leaves the peer usable.

4.7 Package 5: identity, opening, and the wire

Implement empty/active/retired opening states and durable unsent reservations. Exercise persistent seed, bootstrap provider and recipient, ordinary gossip, retirement donor and recipient, and reopen. Keep the memory bookmark path. Restore format and codec-relevant configuration before decoding stored data. Application payload-schema compatibility remains an application obligation. If a decoder or admission-setting change can reject previously accepted bytes, require an explicit streaming validation/migration before enabling gossip from that store. Lazy record checks and a root-format version do not establish compatibility of every stored payload with a new T.

Implement the owner-reviewed wire change for origin tags. Update every leaf encoder, decoder, observer/renderer path, byte-accounting formula, and fixture. Do not claim unchanged memory or stream behavior simply because the new field is fixed-width.

Acceptance:

- Crash before and after reserve, relinquish, first identity-byte write, recipient publication, and completion certificate. No schedule produces overlapping active parties or reuses an externally observed version.
- An ambiguous handoff can strand authority but cannot recover it twice.

- A retired store cannot reopen as active; corrupt storage cannot become Empty; changing universe or store identity is never an implicit repair.
- A crash of all persistent peers followed by reopen requires no new seed. Their state and identity remain usable under the established universe.
- Mixed memory/persistent sessions and all-memory sessions obey the same wire contract and causal-order rules.
- Slow storage, cancellation, one-byte transport buffers, and concurrent reclamation do not introduce a wait cycle.
- Reaccept deliberate wire snapshots through the repository's prescribed process and name the protocol change in the implementing commit.

4.8 Package 6: public observers and lifecycle API

Keep the public facade small. Preserve `Peer<T, Bookmark>` and its memory behavior while developing persistence behind a separate module/facade; decide whether to unify generic parameters only after the internal boundary works. Avoid exposing one omnibus backend trait covering storage, ordering, reclamation, and session scheduling.

Illustrative surface, with names to settle during API review:

```

persistent::Peer::open(store) -> Active(peer) | Empty(store) | Retired(store)
persistent::Peer::seed_in(store)
persistent::Peer::bootstrap_in(store)

rumors.try_send(value).await
rumors.try_redact(version).await
rumors.try_batch(bounded_callback).await
rumors.begin_batch().await -> async staged builder -> commit().await

rumors.snapshot() -> library-owned snapshot
snapshot.try_get(version).await
snapshot.iter() / range(query) -> fallible causal streams
snapshot.unordered() -> explicitly unordered fallible stream

rumors.causal_messages_from(position) -> fallible delivery stream
delivery.after_position() -> serializable local position
observer.checkpoint() -> conservative portable Version

snapshot.release().await
rumors.reclaim().await
peer.close().await

```

Opening and seeding remain distinct operations. An active peer with no messages is still an active peer. Storage errors carry backend context and the stage of failure. Unknown outcomes, definite operation rejection, and maintenance failures after commitment are different categories.

Snapshot release and close do not promise to free objects another live view still holds. Closing prevents new operations and settles outstanding work; the store's exclusive identity lease cannot be released while an old task can still write to it. Specify how surviving snapshots retain read access or delay final store closure; do not silently invalidate a valid snapshot.

Acceptance:

- Cursor resume at every delivery boundary, through redaction-only commits, concurrent publication, empty passes, final None, and reopen.
- A portable checkpoint never advances past an undelivered bounded pass.
- The observer releases storage views while quiet and has no unbounded per-pass backlog.
- Examples show application side effect/cursor ordering without an exactly-once claim, and distinguish local positions from portable versions.

4.9 Package 7: a real store, bounds, and operational documentation

Qualify at least one real adapter against the minimal byte-store contract and run the crash/concurrency suites against it. Choose it by the ease of testing atomic durability and bounded batches on supported targets, not by an assumed engine snapshot facility. Keep additional adapters outside the core crate until they add demonstrated coverage or application value.

Run larger-than-cache tests for bootstrap, sparse and overlapping gossip, large redactions, held snapshots, interrupted sorting, restart cleanup, and sustained small commits competing with a large prepared session.

Report both owned allocations and process-level observations. Include the engine's queues/caches and filesystem cache effects. Process resident memory is not an exact accounting of Rust objects.

Acceptance:

- Controlled memory stays bounded as set and delta sizes increase under fixed value-size and concurrency limits.
- Once external views and private jobs are released and maintenance drains, every remaining object is reachable from the head, every body directory entry is valid, and no zero-work/hold/reservation debris remains.
- Causal-index storage, duplicate transient writes, sort passes, barriers, gate transaction sizes, and conflict work are reported separately.
- Storage reuse is demonstrated under long churn; immediate file shrinkage is not substituted for the reachability test.
- The formal model and maintainer documentation describe the implemented transition system, and the public tutorial includes a durable lifecycle.

4.10 Resource accounting and knobs

Use a budget ledger that covers simultaneous allocations:

```
controlled resident memory <=
    shared decode/body caches and active decoded guards
+ bounded root/operation registry
+ active_sessions * per_session_working_budget
+ bounded local-batch preparation
+ ownership-write and reclamation buffers

value-sized costs = encoded/decoded frontier and party,
                    largest admitted payload/version/record,
                    application T allocations,
                    adapter-owned memory
```

The per-session budget contains the wire window, ingress payloads, traversal frames, all sort buffers, all merge readers and writers, and private batching. Do not reserve a scratch fraction and then allocate that fraction once for each sort space. Do not charge loaded persistent nodes as memory-mode pointers.

Start with explicit knobs for cache bytes, session working bytes, maximum active sessions/views, bounded write bytes/operations, scratch RAM/disk, value-size limits, maintenance work per call and backlog, and retry work. Reject settings too small to perform one legal maximum-fanout ownership step.

The current session budget is useful as a baseline, not proof that it now covers persistence. Select numeric defaults from the real-adapter workload matrix. Avoid committing a 64 MiB cache formula or a one-millisecond reclaim claim before measuring decoded records, version widths, and storage latency.

4.11 Decisions that remain measurements or review choices

The following choices are open to measurement or review. Immutable storage, library-owned snapshots/counts, the minimal engine contract, and preparation without a session-wide write lock are fixed requirements.

Item	Initial recommendation	Evidence required to change it
Final immutable records after origin sorting	Extra address-ordered metadata pass	A faster private-builder design with equally explicit cache/ownership boundaries
Index leaf version	Load the shared message metadata	I/O and version-size measurements favoring duplication
Imported body deduplication	Counted directory acquisition	Measured duplicate-write cost versus directory contention
Retry	Reuse trees; stream admission again; bounded retry work	Region-skipping algorithm plus work counters for selective rebasing
Gate batching	Fixed byte and operation caps	Small-session tail latency and durable barriers per message
Checksum	CRC-32C as an initial fault detector	Chosen fault model and measured alternative, not hostile-peer economics
Public facade	Separate persistence facade initially	Concrete examples showing a simpler unified generic API
Wire version	Deliberate V3 origin-tag vocabulary	Owner review before protocol implementation

Large-session starvation under sustained writes is an unresolved performance obligation, not permission to serialize a large commit. Record an acceptable workload/latency target in package 0, measure it in packages 4 and 7, and revisit the rebase algorithm if it fails.

4.12 Source map

Existing source	Reason to inspect or extend
<code>src/lib.rs</code>	Universe, identity, runtime, payload, and observation contracts
<code>src/tree.rs</code>	Frontier outside the nodes; <code>act</code> and <code>join</code> ; oracle
<code>src/tree/typed/untyped.rs</code>	Compression, sharing, summaries, node representation
<code>src/tree/traverse/join.rs</code>	Base-preferred equality and deletion-honoring traversal
<code>src/tree/mirror/streaming/backend.rs</code>	Materiality boundary and persistent node pricing
<code>src/tree/mirror/streaming/materialized.rs</code>	Schedule, source-set obligations, and wait-graph argument
<code>src/tree/mirror/streaming/remote/adapter/decode.rs</code>	Ingress custody and leaf construction context
<code>src/peer/gossip.rs</code>	Snapshot/fork atomicity, donation gates, absorption, completion
<code>src/batch.rs</code>	Closure runs once, queued actions, local version assignment
<code>src/message.rs</code>	Exact cached bytes, codec and payload admission
<code>src/rumors/causal.rs</code>	Current causal ordering and checkpoint timing
<code>src/bookmark.rs</code>	Identity custody across restart and storage obligations
<code>src/link.rs</code>	Session success, failure, cancellation, transport independence
<code>justfile</code>	Verification recipes and commit gate

Appendix A. Review of the earlier proposal

The earlier proposal, *Persistent storage and a causal index for rumors*, records a design conversation from September 6–7, 2026, with Finch as author of record. It has the right architectural direction: keep the content trie, prepare immutable results concurrently, publish roots atomically, and own snapshot retention in the library. Its difficulties come mainly from assigning some responsibilities to the wrong phase and then claiming bounds that those assignments do not support.

This appendix restates the relevant choices before evaluating them; the earlier document is not required reading. “Source” section numbers identify that document for readers comparing the two. These are design findings, not reports of bugs already present in the library.

A.1 What this plan keeps

- The address-based Merkle trie and its existing reconciliation rules.
- A separate local causal index, with fixed-width keys and sender tags.
- Sequential object IDs, immutable published records, and inline child hashes.
- Preparation outside the ownership gate, using structural sharing.
- Library-managed snapshots, reference counts, and crash-resumable cleanup.
- A byte-storage seam with atomic batches, without engine snapshot requirements.
- Property tests, an independent in-memory merge oracle, crash injection, memory accounting, and explicit wire-format review.

Sequential allocation is a sensible initial locality choice. The claim does not need to be that content addressing is always slower. Similarly, avoiding rank computation is reasonable without claiming no other finite-width ordering scheme can exist under an explicit overflow policy.

A.2 Reference accounting belongs in construction, not publication

Source §§5.2, 5.5, and 8.1 say the last batch sets counts for every new node, and publication reads/increments counts for existing children of rewritten paths. The same sections claim a fixed-size batch and work independent of the update.

Take an import creating a million nodes. Setting their counts in the final batch already requires a million updates. Or change many scattered leaves: the number of shared children referenced by the rewritten branches grows with the changed region. Bounded fanout limits the work **per branch**, not the number of branches in the commit.

The replacement counts every allocated object, including private ones. Construction registers edges incrementally in bounded atomic batches. Publication transfers a small number of root ownership claims. This keeps the intended immutable architecture and removes the contradiction rather than moving large transactions into the engine.

A.3 An owned graph needs roots beyond public snapshots

Source §§4.5 and 8.4 say only snapshots pin storage, while sessions and preparations retain root handles. A raw pointer does not protect storage from a collector just because a future might use it later.

The essential test is simple: a session captures root R , another operation publishes R_2 and reclaims the old head, and the session then reads a child of R . Its lifetime must protect that child. Treat session views, prepared roots, construction roots, and user snapshots as explicit graph owners. Clones can share one in-process owner, as Arc does. The persistent count records ownership of graph edges and roots, not each cheap clone made by a traversal.

A.4 The proposed cache can keep the whole tree resident

Source §4.1 puts a `OnceLock<Arc<Decoded>>` in persistent handles; decoded branches contain child handles. After reading every child, the root can reach every decoded descendant through those strong references. Evicting entries from a separate least-recently-used cache does not free that graph.

An evictable cache also cannot generally return an ordinary borrowed `&Decoded` for the handle's whole lifetime without some pinning arrangement.

Use raw child IDs in decoded records and explicit, bounded loaded-node guards. Keep storage ownership separate from cache ownership. Charge objects held outside the cache as well as cache entries.

A.5 Body counts are not inherently a large-commit bottleneck

Source §8.2 rejects body counts because they would require one count update per message under the publication mutex. That conclusion follows from its choice to defer ownership accounting until publication.

With construction-time accounting, a message's body edge is registered when the message object is built. A snapshot then keeps the body alive through ordinary graph reachability. Freeing a body requires no point lookup in every current or historical tree and no mutable registry scan over an unbounded number of snapshots.

The source's membership-based alternative also needs a race protocol: a "not held anywhere" result must remain valid until the deletion commits, while other sessions can stage or publish the same body. The counted-edge construction transaction states that protocol directly.

Body sharing is still worthwhile, but distinguish bodies of already-authored incoming messages from speculative local sends. Private drafts can temporarily use the same next version for different payloads. They cannot safely share a global version-address directory.

A.6 The staged-body cleanup order leaks bodies

Source §8.3 tests body liveness while the session's own StagedBodies marker still exists. That test must retain the body. The subsequent marker deletion does not include another liveness test.

For a session that aborts after receiving one otherwise-unheld message:

```
delete its private leaf -> its marker says retain body
delete its marker       -> nothing schedules the remaining body for deletion
```

The crash-open sequence has the same issue. Additional bookkeeping can fix it, but counted bodies remove the separate membership/marker mechanism. Releasing the last candidate or message edge naturally schedules the body.

The allocator-chunk scan also checks every allocated record in those chunks for a count, even on a successful session. It cannot have total cost only proportional to rejected messages. Durable construction roots avoid this classification scan entirely.

A.7 A set of pending decrements loses multiplicity

Source §8 uses Free[ptr] as a queue while describing decrements of queued pointers. If two different freed parents each owe a decrement to the same child, putting that child into a set twice records only one obligation.

There are two sound representations: a counted queue of decrement obligations, or immediate atomic decrements with a set containing only objects already at zero. The replacement chooses the latter. It is smaller to explain and gives each zero object one unambiguous deletion transition.

Also define counts during incomplete cleanup. Saying they are correct only after the queue drains leaves the most important crash states unspecified. Count outgoing edges of objects waiting for reclamation until the batch that actually deletes those edges.

A.8 A publication binding must outlive old readers

Source §6.5 deletes Epochs[commit_id] when the group's currently live entry count reaches zero. This table maps a group to its publication number (the source's "epoch"). But an older snapshot or an in-flight session can still hold a leaf from that group and need the binding to produce its tag.

Reference the small binding object from immutable message records. It then lives as long as any current, historical, or private graph needs it, and disappears without retaining a list of redacted messages.

Assign within-group positions once and permit holes. The only value that must wait for publication is the group's publication number. Keeping that uncertainty in one small reservation is clearer than spreading mutable placement through ordinary content records.

A.9 The retry argument overlooks frontiers and candidate scans

Source §7.4 says a candidate's answer changes only below nodes replaced by the intervening commit. Consider a frontier-only advance: the content tree stays empty, but the newer frontier establishes that an incoming candidate has been redacted. Every content pointer can be unchanged while the candidate's admission answer changes.

Rebasing must account for both content changes and frontier changes. A content-identity early stop alone is unsound.

The source also proposes checking each admitted candidate while claiming work depends only on overlap. Even a cheap check for each of a million candidates visits a million candidates. Structural sharing saves many tree reads; it does not automatically skip the outer loop.

Start with a correct streaming rebase and honest cost. Add selective rebasing only with an algorithm that can skip candidate regions, including the effects of frontier-only changes. Do not use a whole-reprepare mutex as the fallback for repeated conflicts. That negates the concurrency the immutable design is intended to preserve.

A.10 Durable identity changes cannot wait for session completion

Source §5.1 identifies the session's final root write as the place party changes become durable, but a donation has already crossed the wire before that final write in the existing lifecycle.

If a donor transmits a party and crashes while its durable head still owns that party, reopening can duplicate the recipient's authority. A perfect content transaction does not prevent that failure.

The current gossip driver (`src/peer/gossip.rs`) already contains the important ordering: snapshot/fork together, and remove donated authority from the bookmark before sending it. Persistence needs an equivalent durable reservation/relinquishment protocol, with the recipient's content and authority published together before completion.

The head also needs the network identifier, store identifier, codec-relevant configuration, and explicit lifecycle state. `party`: bytes alone cannot express an unused store, recoverable unsent retirement, and completed or uncertain retirement safely.

A.11 Causal closure and exact resume need precise statements

Source L-1 says that for an absent cause its version is contained in the frontier. Containment alone does not prove the replica actually incorporated the cause: it also holds after an erroneous partial import advances the frontier and loses the cause's body.

State the invariant over actual emitted messages and their incorporation, and derive it from complete state transitions. The index proof then follows from that invariant and the sender's within-group order. Do not make the containment test itself carry the proof of completeness.

A saved local key gives exact continuation from that key. It does not atomically couple delivery to an application's side effects or cursor store. The supplied "durable position without replay" goal needs this qualification.

Finally, an observer that performs async reads cannot safely walk a root without retaining it somehow. It can retain a bounded page's view and release it while quiet. A portable checkpoint needs a captured frontier/boundary pair; borrowing whichever frontier is newest when a scan finishes can skip an intervening commit.

A.12 Several performance claims should become experiments

The source contains reasonable hypotheses presented as universal facts:

- A large delta, batch action list, or root registry is not bounded merely because the final write buffer has a cap. Every collection needs a bound or a spill path; arbitrary T decoding needs a separate qualification.
- “Two passes” of external merging is insufficient once run count exceeds the square of fanout. All passes and run-directory memory must be covered.
- Comparison sorting takes $O(k \log k)$ comparisons; fixed-width integer keys also admit radix-based sorting. The comparison lower bound is not a lower bound on every possible implementation of these sort keys.
- A cache of encoded top-level records is not the same size as their decoded representation, version summaries, and externally held guards. Nor is arrival-order proximity a guarantee that every causal range prunes well.
- Per-node boxing and lazy decoding are not free on the memory path. The existing decoded payload plus exact serialized bytes is a deliberate cache in `Message (src/message.rs)`, not automatically duplicate waste to remove during backend refactoring.
- A plan cannot infer that a destructor-safety change is unnecessary from holding just the old root. Temporary and losing inputs can contain user values too. Retain tests and inspect the actual implementation.

The implementation packages turn these hypotheses into measurements and acceptance tests. Section 4.11 discusses the unresolved concurrency risk.